

Material Stream Identification System

Technical Report & Pipeline Documentation

Cairo University — Faculty of Computing and Artificial Intelligence

Machine Learning Course — Spring 2025

Executive Summary: This report documents the end-to-end implementation of a feature-based waste material classification system using handcrafted image descriptors and two classical ML classifiers (SVM and k-NN). The final SVM model achieves **79.03% overall accuracy**. All design decisions, experimental comparisons, and failure analyses are documented herein.

1. Motivation & Problem Statement

Automated waste sorting is a critical bottleneck in achieving circular economy goals. Manual sorting is expensive, error-prone, and not scalable. This project addresses the problem of classifying images of waste materials into seven distinct categories using classical machine learning — without relying on deep learning architectures.

The choice of classical ML over deep learning is intentional: handcrafted feature descriptors produce compact, interpretable representations that are computationally feasible on CPU-only hardware and do not require large labeled datasets or GPU infrastructure.

2. Dataset & Classification Categories

The system classifies waste images into six primary recyclable categories plus one synthetic unknown class. The unknown class acts as a safety fallback for low-confidence predictions.

ID	Name	Description	Visual Challenge
0	Glass	Bottles, jars, transparent containers	Transparent — takes on background color
1	Paper	Newspapers, office paper, sheets	Easily confused with cardboard
2	Cardboard	Boxes, packaging, layered cellulose	Texture similar to paper
3	Plastic	Bottles, wrappers, containers	Highly variable shape and color
4	Metal	Cans, aluminum, steel scrap	Shiny surfaces mimic glass
5	Trash	Mixed non-recyclable waste	Most visually inconsistent class
6	Unknown	Synthetic out-of-distribution samples	Synthetic — always correctly rejected

Table 1: Material classes with visual classification challenges

3. System Architecture

The pipeline is implemented as a series of sequential Python scripts, each producing persisted artifacts consumed by the next stage. This design ensures full reproducibility.

Stage	Script	Input	Output
1. Data Preparation	augment.py	Raw dataset images	Augmented train/test splits
2. Unknown Generation	create_unknown.py	Config parameters	Synthetic unknown images
3. Feature Extraction	features.py	Augmented images	X_train.npy, X_test.npy, scaler.pkl
4. Model Training	train.py	Feature arrays (.npy)	svm_model.pkl, knn_model.pkl, best_model.pkl
5. Evaluation	evaluate.py	Saved models + features	Reports, confusion matrices (.png)
6. Inference	realtime_app.py	Webcam feed	Live classification overlay on screen

Table 2: Pipeline stages and their produced artifacts

4. Data Augmentation

Data augmentation is applied exclusively to the training split, never to the test set. Augmenting test data would produce optimistic accuracy estimates that do not reflect real-world performance.

4.1 Augmentation Techniques

Technique	Parameters	Justification
Rotation	-25 to +25 degrees	Waste items appear at any orientation in bins
Horizontal Flip	p=0.5	Mirror symmetry is common in recyclable items
Random Crop + Resize	Scale 0.75–0.95 of original	Simulates partial visibility and distance variation
Brightness/Contrast	alpha=0.75–1.35, beta=-25 to 25	Models varying lighting conditions in real environments
Gaussian Blur	Kernel 3x3 or 5x5	Simulates motion blur and camera defocus
Gaussian Noise	std=5–20	Simulates sensor noise in real-world cameras

Table 3: Augmentation techniques applied to training data only

Each training image undergoes 1–3 randomly selected operations per augmented copy. Each class is padded to a target of 500 training images, ensuring balanced class representation regardless of original dataset size disparities.

4.2 Train/Test Split

Split	Size / class	Augmented?	Purpose
Train	500 images (augmented)	Yes	Model fitting and cross-validation during GridSearchCV

Split	Size / class	Augmented?	Purpose
Test	~20% of original	No	Final unbiased performance evaluation
Unknown Train	500 (synthetic)	N/A	Training the rejection mechanism
Unknown Test	100 (synthetic)	N/A	Evaluating the rejection mechanism

Table 4: Dataset split configuration

5. Feature Extraction

All images are resized to 128x128 pixels before extraction. The pipeline concatenates 8 distinct descriptors into a single 2178-dimensional feature vector per image.

Descriptor	Dims	Captures	Why Chosen
HOG (16x16 cells)	1764	Shape, gradients, edge orientation	Primary shape discriminator; robust to color variation
Color Histograms (BGR, HSV, LAB)	288	Color distribution in 3 spaces	Different materials have distinct color signatures
HSV Color Moments	9	Mean, std, median per channel	Compact color summary; illumination-robust
LBP (Local Binary Pattern)	26	Micro-texture patterns	Differentiates smooth plastic from rough cardboard
GLCM (Gray Co-occurrence)	72	Contrast, energy, homogeneity	Captures surface texture relationships
Hu Moments	7	Shape invariants	Scale/rotation-invariant shape description
Edge Features (Canny)	4	Edge density, contour statistics	Structural complexity of object boundaries
Gabor Filters (4 orientations)	8	Frequency-based texture	Texture at multiple spatial frequencies/angles
Total	2178	Multi-modal representation	—

Table 5: Feature vector composition (total: 2178 dimensions)

5.1 Feature Scaling

After extraction, all features are standardized using StandardScaler (zero mean, unit variance). This is critical for SVM (margin optimization is scale-sensitive) and k-NN (distance calculations are dominated by large-magnitude unscaled features). The scaler is fit on training data only and applied identically to train and test sets.

6. Unknown Class — Rejection Mechanism

A mandatory Unknown class handles out-of-distribution inputs using two complementary strategies:

- **Training-time:** A synthetic unknown class is generated using four generators: random pixel noise, blurred noise, plain solid colors with slight noise, and random geometric shapes. 500 training and 100

test synthetic images are produced.

- **Inference-time:** A confidence threshold of 40% is applied in `realtime_app.py`. If max class probability < 0.40, the output is overridden to Unknown regardless of the model's top prediction.

This explains why Unknown achieves F1=1.00 in evaluation — synthetic noise images are trivially distinct from real material photos in feature space.

7. Model Training & Hyperparameter Tuning

7.1 Support Vector Machine (SVM)

The SVM uses an RBF (Radial Basis Function) kernel, which implicitly maps feature vectors into a higher-dimensional space where a linear hyperplane separates classes. RBF is appropriate here because material boundaries in feature space are non-linear. `class_weight='balanced'` compensates for remaining class imbalance (particularly 'trash' with only 22 test samples). `probability=True` enables confidence scores for the rejection mechanism.

Hyperparameter	Search Space	Best Value	Effect on Model
C (regularization)	1, 3, 10, 30, 100	10	Controls margin width vs. misclassification penalty. Higher C = tighter fit to training data
gamma	scale, 0.01, 0.003, 0.001	scale	RBF kernel width. 'scale' = $1/(n_features * X.var())$, auto-adapts to feature scale
kernel	rbf only	rbf	Non-linear decision boundary in transformed space
CV folds	5 (StratifiedKFold)	—	Preserves class ratio in each fold; more reliable than random splits
Scoring metric	f1_macro	—	Penalizes all classes equally; not biased toward majority classes

Table 6: SVM GridSearchCV configuration and best results

7.2 k-Nearest Neighbors (k-NN)

k-NN classifies each sample by weighted vote among its k nearest neighbors in feature space. Unlike SVM, k-NN has no explicit training phase — it memorizes the training set and computes distances at prediction time. This makes training instant but prediction slower.

Hyperparameter	Search Space	Best Value	Effect on Model
n_neighbors (k)	3, 5, 7, 9, 11	3	Smaller k = more local decisions, sensitive to noise; larger k = smoother boundary
weights	uniform, distance	distance	Distance-weighted: closer neighbors vote more heavily than distant ones
metric	euclidean, manhattan, minkowski	manhattan	Manhattan distance sums absolute differences; more robust in high dimensions than Euclidean
CV folds	5 (StratifiedKFold)	—	Same protocol as SVM for fair comparison

Table 7: k-NN GridSearchCV configuration and best results

8. Experimental Results

8.1 Final Model Performance

Model	Overall Acc.	Macro F1	Best Parameters	CV Score
SVM (RBF)	79.03%	0.7606	C=10, gamma=scale, kernel=rbf	0.8229
k-NN	71.19%	0.6662	k=3, metric=manhattan, weights=distance	0.7741

Table 8: Final model performance on held-out test set

8.2 Per-Class Performance — SVM

Class	Precision	Recall	F1	Support	Analysis
Cardboard	0.88	0.78	0.83	49	Strong — distinct boxy shape and brown texture
Paper	0.81	0.81	0.81	90	Strong — consistent flat white appearance
Plastic	0.76	0.72	0.74	71	Good — color helps but shape is highly variable
Glass	0.74	0.71	0.72	78	Moderate — transparency causes confusion with plastic
Metal	0.60	0.65	0.62	62	Weak — shiny reflective surfaces mimic glass
Trash	0.52	0.73	0.60	22	Weak — visually inconsistent by definition; smallest support
Unknown	1.00	1.00	1.00	100	Perfect — synthetic noise is trivially distinct

Table 9: SVM per-class classification report

8.3 SVM vs k-NN — Detailed Comparison

The SVM outperforms k-NN by a significant margin. This is expected given the high dimensionality of the feature space (2178 dimensions):

- **Curse of dimensionality:** In high-dimensional spaces, Euclidean and Manhattan distances become unreliable — all points tend to be approximately equidistant. This degrades k-NN's core assumption that similar items are 'close' in feature space.
- **SVM's kernel trick:** The RBF kernel maps features to an even higher-dimensional space where a hyperplane cleanly separates classes — a strength specifically in high-dimensional settings.
- **k-NN noise sensitivity:** With 2178 features, many dimensions carry noise. k-NN treats all dimensions equally, polluting distance calculations. SVM's margin optimization is more robust to irrelevant dimensions.

Aspect	SVM	k-NN
Training time	Very slow (kernel optimization over all training points)	Instant (memorizes training set)
Prediction time	Fast (evaluate kernel for test point only)	Slow (compute distances to all training points)

Aspect	SVM	k-NN
High-dim performance	Excellent (RBF kernel trick handles high dimensions)	Poor (curse of dimensionality degrades distances)
Interpretability	Low (decision boundary in kernel space is opaque)	High (can inspect nearest neighbor examples)
Overall accuracy	79.03%	71.19%
Macro F1-score	0.7606	0.6662
Best use case	Complex, high-dimensional classification problems	Low-dimensional, interpretable, fast-prototype tasks

Table 10: SVM vs k-NN comparison across key dimensions

9. Experiments & Ablation Studies

Multiple configurations were tested systematically to attempt accuracy improvements beyond the baseline. All experiments used the same dataset and evaluation protocol.

Experiment	Configuration	SVM Acc.	kNN Acc.	Verdict
Baseline (best)	HOG(16,16), no PCA, full grid search (100 fits, 5 folds)	79.03%	71.19%	Best overall — submitted
Reduced grid search	HOG(16,16), no PCA, 18 fits, 3 folds	79.03%	65.47%	Same SVM acc, much faster
PCA on baseline	HOG(16,16) + PCA(95%): 2178 → 550 dims	79.03%	65.25%	No SVM gain; k-NN hurt by PCA
Finer HOG, no PCA	HOG(8,8), 8514 dims, no PCA	73.52%	65.89%	Worse — too noisy for SVM
Finer HOG + PCA	HOG(8,8) + PCA(95%): 8514 → ~600 dims	73.09%	58.05%	Worst — fine HOG + PCA incompatible

Table 11: Ablation study — all tested configurations

9.1 Why Finer HOG (8,8) Hurt Performance

- **More noise than signal:** Finer cells capture micro-texture details that are not class-discriminative for waste materials. A bottle and a can both have fine horizontal edges — coarser (16,16) cells better capture the overall silhouette shape that distinguishes them.
- **SVM kernel degradation:** With 8514 dimensions, $\gamma = 1 / (8514 * X.var())$ becomes extremely small, making the RBF kernel nearly linear and reducing its ability to model non-linear class boundaries.

9.2 Why PCA Helped Speed But Not Accuracy

PCA (95% variance) compressed 2178 features to ~550 principal components. For SVM this had no effect on accuracy — the original features were already informative enough and SVM was robust to noise dimensions. For k-NN, PCA changed distance relationships in ways that degraded nearest-neighbor quality: PCA maximizes variance, not class separability, so the new distances were less meaningful for classification.

10. Why 0.85 Accuracy Was Not Achieved

The project requirement specifies a minimum validation accuracy of 0.85 on the six primary classes. The system achieves 79.03% overall. This gap reflects fundamental limitations of classical ML on this problem — not a failure of implementation.

10.1 Data-Level Limitations

- **Intra-class visual diversity:** The 'trash' class by definition includes everything that does not fit other categories — a banana peel and a styrofoam cup are both 'trash' but share no visual features. No feature descriptor can learn a coherent boundary for a semantically incoherent class.
- **Inter-class visual similarity:** Metal cans and glass bottles are both cylindrical and reflective. Plastic and glass bottles are nearly identical in shape. These pairs consistently produce the lowest F1 scores (metal: 0.62, glass: 0.72).
- **Lighting and background variation:** Handcrafted features extract global statistics sensitive to illumination, unlike deep CNN features which learn illumination-invariant representations automatically.

10.2 Feature-Level Limitations

- **Handcrafted features are not learned:** HOG, LBP, GLCM, and Gabor capture what engineers think is important, not what is actually discriminative for this specific dataset. Deep networks discover task-specific features directly from data.
- **No semantic understanding:** The feature vector captures texture, color, and edge statistics but has no concept of 'this is a cylindrical transparent container.' Material identity is a semantic concept handcrafted features cannot fully encode.

10.3 Model-Level Limitations

- **SVM capacity limit:** Even with RBF, the SVM finds a single decision boundary per class pair. Real material boundaries in feature space may be highly fragmented and non-convex.
- **k-NN curse of dimensionality:** Distance-based classification fundamentally degrades in 2178-dimensional space, as analyzed in Section 8.3.

10.4 What Would Achieve 85%+

Approach	Expected Gain	Complexity
Fine-tuned CNN (ResNet-18, MobileNet)	+10–15%	High — requires GPU, PyTorch or TensorFlow setup
Transfer learning (ImageNet pre-trained)	+8–12%	High — but less training data needed
Larger, more diverse dataset	+3–5%	Medium — data collection and labeling effort
Ensemble of multiple classifiers	+1–3%	Low — can build directly on current code
Learned feature embeddings (deep)	+5–8%	Medium — replaces handcrafted feature extractor

Table 12: Approaches likely to achieve the 0.85 accuracy target

11. Real-Time Deployment

The best-performing SVM model is deployed in a live webcam application using OpenCV. Frames are processed in real-time with classification results shown as an overlay.

Component	Implementation	Detail
Frame capture	cv2.VideoCapture(0)	Default webcam; index configurable for multi-camera setups
Region of interest	300x300 center crop	Focuses model on object held in front of camera
Prediction frequency	Every 10 frames	Balances UI responsiveness with CPU load
Feature pipeline	extract_features() + scaler.transform()	Identical pipeline to training — no re-fitting at inference
Confidence threshold	0.40 (40%)	Below threshold → output overridden to 'Unknown'
Display overlay	cv2.putText() labels	Shows class, confidence %, top-3 predictions, and FPS

Table 13: Real-time deployment implementation details

12. Reproducibility Notes

All random operations use SEED=42. Two sources of non-determinism were identified:

- **Path.iterdir() ordering:** Python's pathlib returns files in filesystem order, which differs between OS/machines. Even with the same seed, train/test splits can vary by 1–7% accuracy across machines. Fix: use sorted(path.iterdir()) consistently.
- **n_jobs=-1 parallelism:** GridSearchCV with parallel jobs produces non-deterministic ordering across machines with different core counts, potentially selecting different best hyperparameters. Fix: use n_jobs=1 for fully reproducible results.

For submission, saved .npy feature files and .pkl model files are committed to the repository so evaluate.py produces consistent results on any machine.

13. Future Work & Recommendations

- **Deep learning baseline:** A fine-tuned MobileNetV3 or EfficientNet-B0 on the same dataset would serve as a meaningful comparison and likely exceed the 0.85 accuracy target.
- **Better unknown handling:** Use real out-of-distribution waste images (e.g., electronic waste, hazardous materials) instead of synthetic noise for a more realistic rejection mechanism.
- **Proper three-way split:** Add a separate validation set for model selection, keeping the test set truly held-out for final unbiased reporting.
- **Feature selection:** Apply mutual information or ANOVA F-test to identify and remove the least discriminative dimensions before training.
- **Class-specific augmentation:** Apply heavier augmentation to weak classes (trash, metal) and lighter augmentation to strong classes (cardboard, paper) to better balance difficulty.

Appendix A: Dependencies

Package	Purpose in Pipeline
scikit-learn	SVM, k-NN, GridSearchCV, StandardScaler, LabelEncoder, PCA, metrics
opencv-python (cv2)	Image I/O, augmentation transforms, feature extraction, webcam capture
scikit-image (skimage)	HOG, LBP (local_binary_pattern), GLCM (graycomatrix, graycoprops)
numpy	Array operations, feature vector concatenation and storage (.npy)
joblib	Model serialization and deserialization (.pkl files)
matplotlib + seaborn	Confusion matrix heatmap visualization and report figure generation
tqdm	Progress bars during feature extraction loops

Table 14: Python dependencies and their roles

Appendix B: Repository File Structure

```
material-stream-classifier/
├── dataset/ # Raw original images (per-class subfolders)
├── data/
│   ├── augmented/ # Processed train/test splits with augmented images
│   ├── train/ # Per-class training images (500 each after augmentation)
│   └── test/ # Per-class test images (raw, no augmentation)
├── features/ # X_train.npy, X_test.npy, y_train.npy, y_test.npy,
│   └── # scaler.pkl, label_encoder.pkl
├── models/ # svm_model.pkl, knn_model.pkl, best_model.pkl
├── reports/ # confusion_matrix_svm.png, confusion_matrix_knn.png,
│   └── # classification_report_svm.txt, classification_report_knn.txt
├── src/
│   ├── config.py # All configuration constants (paths, IMG_SIZE, SEED)
│   ├── utils.py # Shared helper functions (collect_image_paths, save_image)
│   ├── augment.py # Step 1: data preparation and augmentation
│   ├── create_unknown.py # Step 2: synthetic unknown class generation
│   ├── features.py # Step 3: feature extraction and scaling
│   ├── train.py # Step 4: GridSearchCV training for SVM and k-NN
│   ├── evaluate.py # Step 5: evaluation, reports, confusion matrices
│   ├── realtime_app.py # Step 6: live webcam inference application
│   └── test_single_image.py # Utility: single image classification from CLI
```